



Instituto Tecnológico y de Estudios Superiores de Monterrey

Escuela de Ingeniería y Ciencias

Ingeniería en Ciencia de Datos y Matemáticas

Etapas 1 y 2: Gestor de Identidades y Firma de Correos

Equipo de Trabajo 2 - Grupo 603:

Valeria Arciga Valencia - A01737555

Henning Arvid Ladewig - A01831983

Alberto Boughton Reyes - A01178500

Ximena Montes Bautista - A01737949

Diego Octavio Arias Incháustegui - A00838285

Fernando Daniel Saucedo Hernández - A01385490

Profesores del bloque:

Raúl Gómez Muñoz, Daniel Otero Fadul,
Alberto Francisco Martínez Herrera, Pedro Leonel Olaya Trejos

Socio Formador:

Casa Monarca Ayuda Humanitaria al Migrante A.B.P.

Monterrey, Nuevo León

14 de junio de 2026

Índice general

Abstract	4
1. Introducción	5
2. Marco de Referencia	6
2.1. Gestión de Identidades Digitales y Control de Acceso (IAM)	6
2.2. Criptografía y Confidencialidad de la Información	6
2.3. Metodología de Desarrollo Seguro (DevSecOps) y Control de Versiones	7
2.4. Arquitectura Tecnológica y Bases de Datos	7
2.5. Contexto Legal y Normativo	7
2.6. Autenticación con Tokens Web y Gestión de Sesiones	8
3. Desarrollo	8
3.1. Propuesta	8
3.1.1. Visión General	8
3.1.2. Justificación	9
3.2. Gestión de identidades y política de control de acceso	10
3.3. Desarrollo de componentes BackEnd	12
3.4. Desarrollo de FrontEnd	15
3.5. Conexión y flujos de operación	18
3.5.1. Flujos de operación principales	18
4. Resultados	24
5. Conclusiones y Trabajo Futuro	26
Trabajo Futuro	27
Referencias	30

Índice de cuadros

3.1. Matriz de compatibilidades e incompatibilidades tecnológicas del entorno de hosting. (elaboración propia).	9
3.2. Matriz de permisos CRUD sobre el padrón de usuarios (elaboración propia).	11
3.3. Matriz de capacidades criptográficas y firma institucional por nivel de acceso (elaboración propia).	11
3.4. Estructura de Enums y valores permitidos para el control operativo (elaboración propia). . . .	12
3.5. Plantillas que componen la capa de presentación del sistema (elaboración propia).	16
4.1. Matriz de casos de prueba y enlaces de evidencia para los módulos de Autenticación, Usuarios, Certificados y Correos firmados. (elaboración propia).	25

Índice de figuras

3.1. Diagrama de flujo: registro de nuevos usuarios con generación automática de contraseña temporal (elaboración propia).	19
3.2. Diagrama de flujo: registro de un certificado X.509 emitido externamente por la CA interna (elaboración propia).	20
3.3. Diagrama de flujo: consulta y filtrado de certificados mediante el endpoint GET /api/certs (elaboración propia).	21
3.4. Diagrama de flujo: autenticación de usuario con discriminación de primer acceso (elaboración propia).	22
3.5. Diagrama de flujo: firma de un correo institucional y generación condicional del token efímero de verificación (elaboración propia).	23

Abstract

Este proyecto presenta el diseño e implementación de un sistema integral de gestión de identidades y comunicaciones institucionales firmadas para Casa Monarca Ayuda Humanitaria al Migrante A.B.P., organización de la sociedad civil que brinda apoyo legal, humanitario y educativo a personas migrantes en situación de vulnerabilidad en Nuevo León, México. La solución, desarrollada bajo las restricciones de un entorno de alojamiento compartido, fue construida con Python (Flask), la biblioteca `cryptography` y TinyDB como capa de persistencia ligera.

El sistema adopta una arquitectura cliente-servidor de tres capas e integra tres módulos funcionales principales. En primer lugar, un módulo de gestión de identidades que implementa criptografía de clave pública (RSA de 2048 bits como mínimo, configurable hasta 4096 bits), certificados digitales X.509 firmados con SHA-256 y un esquema de Autoridad Certificadora interna operada por el personal administrativo de la organización. En segundo lugar, un módulo de autenticación basado en tokens JWT (JSON Web Tokens) firmados con HS256 y contraseñas resguardadas mediante el algoritmo de hash `bcrypt`: el sistema asigna contraseñas temporales generadas mediante un generador criptográficamente seguro y fuerza su cambio en el primer inicio de sesión. En tercer lugar, un módulo de firma digital de correos institucionales que permite a los usuarios con privilegios de coordinación o administración emitir comunicaciones oficiales firmadas criptográficamente, garantizando los principios de integridad, autenticidad y no repudio.

Un modelo de Control de Acceso Basado en Roles (RBAC) con cuatro niveles jerárquicos rige todas las operaciones del sistema, y restringe las capacidades de firma criptográfica al personal directivo. Para la verificación de correos por destinatarios externos, el sistema implementa un mecanismo de enlaces efímeros: cada mensaje dirigido fuera de la organización incluye un token único de un solo uso con vigencia de 24 horas, accesible desde cualquier navegador sin necesidad de credenciales ni software cliente especializado, alineándose con un principio de máxima accesibilidad para terceros.

El sistema fue diseñado en cumplimiento de la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP) y facilita el ejercicio de los Derechos ARCO. Las pruebas funcionales confirmaron la correcta operación de los flujos criptográficos, el control de acceso jerárquico, la integridad de las firmas digitales y los mecanismos de verificación pública. La arquitectura modular y desacoplada permite una futura migración hacia un motor de base de datos relacional y la incorporación de funcionalidades adicionales como autenticación multifactor, persistencia de listas de revocación y mecanismos avanzados de protección de credenciales criptográficas.

1. Introducción

La migración forzada es un fenómeno a nivel global en el que millones de personas se ven obligadas a abandonar sus lugares de origen en búsqueda de seguridad, estabilidad y mejores oportunidades. Dentro de este contexto, Casa Monarca Ayuda Humanitaria al Migrante A.B.P., fungiendo como Organización Socio Formadora (OSF) dentro de la unidad de formación Uso de álgebras modernas para seguridad y criptografía, desempeña un papel esencial al brindar apoyo a migrantes en situación de vulnerabilidad mediante servicios como la asesoría legal, la ayuda humanitaria, la orientación laboral y el apoyo educativo.

Para llevar a cabo estas labores de asistencia, la organización requiere manejar información altamente sensible, incluyendo datos personales, documentos legales, constancias y registros médicos o jurídicos. Sin embargo, al igual que muchas organizaciones, Casa Monarca se enfrenta a limitaciones de recursos y barreras técnicas que dificultan la implementación de sistemas de seguridad informática avanzados, lo que expone esta información a un estado de vulnerabilidad. Adicionalmente, una porción significativa de la operación institucional depende del intercambio de comunicaciones oficiales hacia el exterior (con autoridades gubernamentales, donantes, instituciones aliadas y beneficiarios), comunicaciones cuya autenticidad debe poder garantizarse ante terceros que no necesariamente cuentan con infraestructura técnica propia. Al mismo tiempo que los retos técnicos que exige la situación problema, es importante que se cumpla con la legislación mexicana, la cual exige a toda organización garantizar el manejo y la protección adecuada de los datos personales.

En respuesta a esta problemática, el presente reto tiene como objetivo central desarrollar y proporcionar a Casa Monarca Ayuda Humanitaria al Migrante A.B.P. herramientas tecnológicas robustas que permitan garantizar que todos sus procesos digitales operen bajo los principios fundamentales de Confidencialidad, Integridad y Disponibilidad, logrando así beneficiar sus operaciones. Específicamente, la solución busca asegurar el estricto cumplimiento de la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP) y facilitar el ejercicio de los Derechos ARCO (Acceso, Rectificación, Cancelación y Oposición) para los beneficiarios de la organización. Asimismo, el sistema está diseñado para dotar de veracidad, autenticidad y no repudio a los documentos y comunicaciones que emita la OSF frente a terceros.

Para alcanzar estas metas, el proyecto propone la creación de un sistema integral que articula tres capacidades complementarias. En primer lugar, una plataforma de gestión de identidades basada en certificados digitales X.509, con capacidades para dar de alta, revocar y rastrear de forma segura los accesos a la información. En segundo lugar, un módulo de autenticación de usuarios basado en tokens criptográficos y resguardo de contraseñas mediante funciones hash robustas, que garantiza la trazabilidad individual de cada acción ejecutada en el sistema. En tercer lugar, un módulo de firma digital de correos institucionales que permite emitir comunicaciones oficiales con valor probatorio, incorporando un mecanismo de verificación pública accesible para destinatarios externos sin requerimientos técnicos adicionales.

Finalmente, para garantizar el éxito y la adopción operativa de la herramienta, la implementación se ha diseñado para ser totalmente transparente hacia los usuarios y colaboradores. El sistema busca no complicar la operación diaria ni exigir conocimientos técnicos avanzados al personal, enfocándose en mejorar los tiempos de atención y la productividad general. De esta forma, la solución entregada se proyecta como una plataforma fácil de usar, adaptada a los recursos actuales de la organización.

2. Marco de Referencia

Para el desarrollo del sistema de gestión de identidades, envío de correos y protección de datos para la organización Casa Monarca, fue fundamental analizar tanto el contexto legal mexicano como las tecnologías y metodologías de ciberseguridad actuales. Este capítulo detalla los conceptos, estándares y soluciones existentes en la literatura que sirvieron como base para el diseño y la toma de decisiones arquitectónicas del proyecto.

2.1. Gestión de Identidades Digitales y Control de Acceso (IAM)

El manejo de identidades digitales y el control de acceso (IAM) constituye un desafío tecnológico abordado a nivel global mediante diversas plataformas consolidadas. La identidad digital se define como "la representación única de un sujeto que participa en actividades en línea"[Grassi et al., 2017]. Su correcta gestión es un factor crítico, ya que, al tratarse de información altamente personal y sensible por razones socio políticas, una vulneración podría resultar en severos daños e impactos, tales como divulgación no autorizada de información confidencial, seguridad personal vulnerada o infracciones civiles o penales [Grassi et al., 2017].

En la industria, soluciones de código abierto como Keycloak ofrecen arquitecturas robustas para la gestión de usuarios, roles y autenticación multifactor en entornos empresariales. Esta plataforma funciona como un intermediario utilizando protocolos estándar como OpenID Connect y SAML 2.0. Además, facilita el Single Sign-On (SSO) y gestiona la seguridad mediante el intercambio de JSON Web Tokens (JWT), garantizando la integridad en cada transacción. Sin embargo, la implementación de estos sistemas preconfigurados a menudo exige una infraestructura dedicada que excede las capacidades del alojamiento compartido que actualmente posee Casa Monarca. Por este motivo, la propuesta opta por un desarrollo adaptado a la medida de la Organización Socio Formadora. En este sentido, es imperativo establecer mecanismos que "limiten el acceso a los recursos de un sistema únicamente a los usuarios autorizados" [Stallings, 2017]. Adicionalmente, como señalan [Zúñiga and Hecht, 2023], los sistemas modernos requieren "mecanismos de autorización dinámicos y granulares que permitan asignar privilegios únicamente a los recursos necesarios y durante periodos de tiempo limitados".

2.2. Criptografía y Confidencialidad de la Información

En el núcleo del sistema, el resguardo de la confidencialidad y la autenticidad depende del uso de algoritmos de criptografía asimétrica y funciones de resumen criptográfico. Específicamente, el sistema se apoya en el algoritmo RSA [Rivest et al., 1978] para la generación de pares de llaves, la firma digital de certificados y la firma de comunicaciones institucionales. De manera complementaria, se integra el uso de funciones resumen, particularmente el estándar SHA-256, para verificar la integridad de los datos almacenados y en tránsito [Zong, 2025]. Tras evaluar los requerimientos de la organización, se determinó que no se establecerán distinciones criptográficas ni esquemas de protección diferenciados por edad, garantizando que toda la información de la población atendida reciba el nivel máximo de protección posible sin sobrecargar el servidor.

Para la firma digital de mensajes, el sistema adopta el esquema RSA-PKCS1v15 en combinación con SHA-256, una combinación ampliamente utilizada en estándares como TLS, S/MIME y la propia firma de certificados X.509 [Barker, 2020]. Este esquema garantiza que cualquier alteración del contenido firmado, por mínima que sea, invalide la firma durante la verificación, asegurando integridad y no repudio. Complementariamente, para el resguardo de credenciales de autenticación, el sistema implementa el algoritmo de hash adaptativo bcrypt, diseñado específicamente para almacenamiento seguro de contraseñas. A diferencia de funciones hash genéricas como SHA-256, bcrypt incorpora un parámetro de costo computacional configurable que se ajusta a la capacidad del hardware moderno, dificultando ataques de fuerza bruta incluso ante el compromiso de la base de datos [Provos and Mazières, 1999].

Para la implementación práctica, se emplean bibliotecas criptográficas establecidas como OpenSSL y frameworks en Python, los cuales son ampliamente usados en el desarrollo de algoritmos modernos. Estas

herramientas permiten la implementación robusta de "generación de pares de claves, firmas digitales y validación de certificados en aplicaciones personalizadas"[Cryptography Project, 2026].

2.3. Metodología de Desarrollo Seguro (DevSecOps) y Control de Versiones

Para garantizar que el ciclo de vida del software sea seguro desde sus cimientos, el desarrollo se rige bajo la metodología DevSecOps. Esta práctica garantiza la integración continua de la seguridad desde las fases iniciales del diseño, permitiendo mitigar vulnerabilidades a través de herramientas de análisis preventivo [Mohammed et al., 2025, Nadipalli, 2023].

A nivel organizativo, la gestión de cambios se realiza mediante herramientas de control de versiones. Se implementa Git porque "es un sistema de control de versiones distribuido diseñado para gestionar todo tipo de proyectos, desde los más pequeños hasta los más grandes, con rapidez y eficiencia" [Chacon and Straub, 2014]. Esto se complementa con GitHub, plataforma que "con herramientas basadas en inteligencia artificial, pruebas de seguridad integradas y una integración perfecta, presta apoyo a los equipos desde el primer commit hasta el desarrollo empresarial" [GitHub, Inc., 2026], facilitando el trabajo colaborativo de manera clara y eficiente.

2.4. Arquitectura Tecnológica y Bases de Datos

La solución se ha modelado bajo una arquitectura cliente-servidor. El lenguaje de programación Python es la opción seleccionada para el backend debido a su alta compatibilidad con bibliotecas criptográficas y su amplio uso en el desarrollo de sistemas de seguridad [OpenSSL Project, 2023]. Para el manejo de la autenticación de usuarios, el sistema incorpora bibliotecas estándar de la industria que son nativamente compatibles con el entorno de despliegue: PyJWT para la emisión y validación de tokens JWT, y bcrypt para el resguardo seguro de contraseñas. Estas decisiones permiten implementar mecanismos modernos de control de sesiones sin requerir infraestructura adicional como servidores de identidad federada. Finalmente, para la persistencia de datos, el sistema se apoya en una base de datos relacional para gestionar integralmente el registro de usuarios, las listas de revocación y la auditoría. MySQL destaca como una solución idónea gracias a su robusto control de concurrencia, mecanismos de seguridad integrada y compatibilidad nativa con los servidores del socio formador. Esta tecnología permite "administrar los privilegios de acceso mediante un sistema de cuentas de usuario y roles, poniendo a disposición del desarrollador un modelo de control de acceso flexible y seguro"[Oracle Corporation, 2024].

2.5. Contexto Legal y Normativo

El manejo de información perteneciente a poblaciones vulnerables, como las personas en situación de migración, trasciende los requerimientos puramente técnicos e impone obligaciones legales rigurosas. En el marco jurídico mexicano, la recolección, resguardo y procesamiento de esta información está regulada de manera estricta por la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP). Esta legislación establece que toda organización (incluidas las asociaciones civiles y ONGs) debe garantizar un tratamiento legítimo, controlado e informado de los datos, a efecto de asegurar la privacidad y el derecho a la autodeterminación informativa de las personas [Cámara de Diputados del H. Congreso de la Unión, 2010].

Uno de los pilares de esta ley es la obligación de proporcionar mecanismos ágiles, gratuitos y transparentes para que los titulares de la información puedan ejercer sus Derechos ARCO (Acceso, Rectificación, Cancelación y Oposición). La arquitectura de gestión de identidades digitales propuesta para Casa Monarca responde directamente a esta exigencia legal. Al sentar las bases para la emisión y gestión del ciclo de vida de los certificados digitales, el sistema permite aislar compromisos de seguridad e inhabilitar accesos de manera trazable, garantizando el principio de responsabilidad y facilitando la Cancelación u Oposición.^{al} tratamiento de datos cuando el beneficiario o voluntario así lo solicite.

Adicionalmente, el proyecto aborda la necesidad de otorgar validez y no repudio a las constancias y documentos emitidos por la Organización Socio Formadora. La implementación de firmas y certificados basados en el estándar internacional X.509 dota de integridad y autenticidad a las operaciones electrónicas. Esto se alinea con los principios de fiabilidad y equivalencia funcional de los mensajes de datos establecidos en el Título Segundo del Código de Comercio y normas oficiales complementarias sobre conservación de mensajes de datos [Secretaría de Economía, 2017], asegurando que las herramientas tecnológicas entregadas a Casa Monarca cuenten con un respaldo probatorio ante terceros.

2.6. Autenticación con Tokens Web y Gestión de Sesiones

Para la gestión de sesiones autenticadas, el sistema adopta el estándar de JSON Web Tokens (JWT), definido formalmente en el RFC 7519 [Jones et al., 2015]. Un JWT es una cadena codificada en Base64URL que encapsula tres componentes: un encabezado que declara el algoritmo de firma utilizado, un conjunto de afirmaciones (*claims*) sobre el usuario y la sesión, y una firma criptográfica que protege la integridad del token. Esta estructura permite que el servidor verifique la autenticidad de cada solicitud sin necesidad de mantener estado de sesión en memoria o consultar la base de datos, lo cual reduce la carga operativa en entornos de alojamiento compartido con recursos limitados.

El sistema implementa el algoritmo de firma HMAC-SHA256 (HS256), que utiliza una llave secreta compartida del lado del servidor para firmar y verificar los tokens. A diferencia de algoritmos asimétricos como RS256, HS256 ofrece rendimiento superior y simplicidad de implementación, ventajas especialmente relevantes cuando todas las verificaciones de token ocurren dentro del mismo servidor que las emite [Jones et al., 2015]. Como contrapartida, este esquema requiere una gestión estricta de la llave secreta, la cual debe permanecer aislada del repositorio de código fuente y rotarse periódicamente. El sistema aborda esta preocupación generando la llave de forma aleatoria durante la configuración inicial y aislándola en un archivo de variables de entorno explícitamente excluido del control de versiones.

El uso de JWT se complementa con un mecanismo de cambio obligatorio de contraseña en el primer inicio de sesión, alineado con las recomendaciones del NIST SP 800-63B [Grassi et al., 2017] sobre gestión del ciclo de vida de credenciales. Este patrón mitiga el riesgo asociado a las contraseñas temporales generadas durante el alta de usuarios, asegurando que ninguna credencial preasignada por el sistema permanezca activa indefinidamente.

3. Desarrollo

En el presente capítulo se documentan a detalle las acciones realizadas para dar solución a la problemática planteada por la Organización Socio Formadora (OSF). A continuación, se expone la propuesta de solución, la arquitectura del sistema, la justificación de las decisiones tecnológicas adoptadas, así como el desglose de la implementación de los componentes del servidor (BackEnd), del cliente (FrontEnd) y la interconexión entre los mismos.

3.1. Propuesta

3.1.1. Visión General

Para resolver la problemática integral de Casa Monarca, descrita en el capítulo de Introducción, se propuso la construcción de un sistema que articula tres módulos funcionales complementarios: la gestión de identidades digitales mediante certificados X.509, la autenticación segura de usuarios mediante tokens JWT y contraseñas resguardadas con `bcrypt`, y la firma criptográfica de comunicaciones institucionales con verificación pública accesible para destinatarios externos. Esta arquitectura modular abarca desde el personal directivo hasta los colaboradores operativos, segmentando capacidades según el nivel jerárquico de cada usuario.

El sistema fue diseñado bajo un esquema de arquitectura cliente-servidor de tres capas (*three-tier architecture*), lo cual permite una separación lógica, modular y escalable de las responsabilidades operativas:

- **Capa de Presentación (FrontEnd):** Una interfaz web responsiva, orientada a ofrecer un panel administrativo intuitivo para la gestión y monitoreo del ciclo de vida de las identidades, junto con vistas especializadas para autenticación, cambio obligatorio de contraseña, redacción de correos firmados y verificación pública de mensajes por terceros.
- **Capa de Lógica (BackEnd / API):** Una interfaz de programación de aplicaciones (API REST) responsable de procesar las reglas de negocio, validar los datos de entrada, ejecutar las operaciones criptográficas de firma y verificación, gestionar las sesiones autenticadas mediante JWT y aplicar el control de acceso jerárquico sobre cada operación.
- **Capa de Persistencia de Datos:** Almacenamiento diseñado de manera desacoplada para facilitar el despliegue inicial del Producto Mínimo Viable (MVP) y permitir una migración transparente hacia un motor relacional robusto en etapas posteriores.

3.1.2. Justificación

El diseño de la propuesta se fundamentó en un análisis exhaustivo de los requerimientos operativos de la organización y sus restricciones de infraestructura física. En particular, se evaluaron las capacidades del servicio de alojamiento compartido actual (*HostGator*) con el que cuenta Casa Monarca.

Para determinar la viabilidad del proyecto, se elaboró una matriz de compatibilidad (Cuadro 3.1), la cual expone las herramientas soportadas por el servidor y las incompatibilidades notables que restringen el uso de tecnologías empresariales preconfiguradas.

Categoría	Herramientas y Tecnologías Compatibles	Incompatibilidades Notables
Lenguajes y Frameworks	PHP: 8.0, 8.1, 8.2. Python: 3.6. Frameworks: Laravel (10 y 11), Django (solo hasta 3.2), Ruby on Rails, CakePHP. Otros: JavaScript, JSON, FastCGI, Perl 5, cURL, módulos PECL.	Node.js, Java (JSP/Tomcat), ASP.NET (todas las versiones), Zend Framework, PHP 5 y 7.
Bases de Datos	Soportadas: MySQL 5.7 (CentOS) o MySQL 8.0 (AlmaLinux), MySQLi, PDO (pdo_mysql).	PostgreSQL, MariaDB, Microsoft SQL Server, Oracle, MongoDB.
CMS y E-commerce	Gestores: WordPress (y WP MU), Drupal, Joomla (hasta 5.5.2). E-commerce: Magento (2.3), PrestaShop (1.7 y 8), OpenCart. E-learning: Moodle (hasta 4.1).	Versiones más recientes o no especificadas de Joomla y Moodle requerirán VPS.
Paneles y Servicios	Paneles: cPanel, WHM, Softaculous (instalador de apps). Accesos: FTP (Cyberduck), SSH2, WebDisk, Tareas Cron. Correo: Webmail (RoundCube), IMAP, POP3, SMTP, Exim.	Plesk, Docker, VPN, Servidores de juegos, Terminales como KVM o Remote Desktop.
Seguridad y SSL	Certificados: SSL Gratis (Let's Encrypt) para todos los dominios, Wildcard SSL, OpenSSL. Antispam: SpamAssassin.	SSL compartido, Certificación HIPAA compliance, PCI Compliance.
Otras Herramientas	Git (Cliente), SVN (Cliente), AWstats, ImageMagick, Archivos comprimidos (.zip, .tar.gz), Ajax, HTML.	FFmpeg, Live streaming, Load balancing, Redis/Memcached, XAMPP.

Cuadro 3.1: Matriz de compatibilidades e incompatibilidades tecnológicas del entorno de hosting. (elaboración propia).

El análisis de esta matriz determinó directamente las decisiones de ingeniería para el Producto Mínimo Viable (MVP). Al presentarse incompatibilidades con motores relacionales como PostgreSQL y entornos de ejecución como Node.js, el equipo optó por un desarrollo a la medida bajo el siguiente stack tecnológico:

- **Lenguaje y Framework Base:** Se seleccionó Python 3.6 (versión máxima soportada nativamente) utilizando el micro-framework Flask (v. 1.1.2) por su bajo costo computacional e ideal para el despliegue de prototipos funcionales y APIs con alta velocidad. A diferencia de arquitecturas más pesadas, Flask proporcionó la flexibilidad necesaria sin imponer consumo excesivo de memoria. Adicionalmente, la extensión Flask-RESTX facilitó la estandarización de endpoints, la validación automática de esquemas de datos y la autogeneración de documentación interactiva a través de Swagger UI.
- **Motor Criptográfico Estandarizado (cryptography):** Para el núcleo de seguridad, se descartaron alternativas de bajo nivel en favor de la biblioteca cryptography (v. 3.4.7), consolidada como el estándar en Python. Esta selección permitió la generación eficiente de llaves asimétricas RSA (con tamaños superiores a los 2048 bits recomendados por el NIST), la estructuración de los certificados X.509, la firma digital de correos institucionales mediante RSA-PKCS1v15 con SHA-256, y la verificación de cadenas de confianza, todo ello operando independientemente del servidor web.
- **Autenticación y Gestión de Sesiones:** Para el módulo de autenticación, se incorporaron las bibliotecas PyJWT (v. 1.7.1) para la emisión y validación de tokens JSON Web Token bajo el algoritmo HS256, y bcrypt (v. 3.1.7) para el resguardo seguro de contraseñas mediante hashing adaptativo con factor de costo 12. La llave secreta para la firma de tokens se gestiona mediante python-dotenv (v. 0.20.0), aislándola del repositorio de código fuente en un archivo .env generado durante la configuración inicial del entorno.
- **Base de Datos (TinyDB):** Para la fase de validación técnica (MVP), se implementó TinyDB (v. 4.4.0), una base de datos ligera basada en archivos JSON que no requiere dependencias externas. Se puede destacar que el diseño de la capa de datos (db.py) se programó con un grado máximo de desacoplamiento, lo que garantiza que, cuando el sistema escale hacia un entorno de producción con bases relacionales completas, la API y el motor criptográfico no requerirán reestructuración.
- **Infraestructura de Llaves Públicas:** Considerando el alcance del proyecto y el contexto corporativo interno de la OSF, se implementó una cadena de confianza fundamentada en una Autoridad Certificadora interna operada por el personal administrativo, con generación de certificados mediante herramientas de línea de comandos ejecutadas localmente por el administrador. Este enfoque garantiza que las llaves privadas de los usuarios nunca transiten por el servidor central.
- **Firma Digital en el Navegador:** Para la firma de correos institucionales, se integró la biblioteca JavaScript jsrsasign en la capa de presentación, lo que permite calcular las firmas RSA-PKCS1v15 directamente en el navegador del firmante utilizando su llave privada local. Esta arquitectura asegura que la llave privada nunca se transmita al servidor, alineándose con el principio criptográfico de no centralización de secretos.
- **Desarrollo de FrontEnd:** Se determinó no integrar bibliotecas modernas de renderizado como React o Angular. Estos marcos de trabajo exigen pasos de compilación complejos, lo que incrementaría drásticamente la dificultad de mantenimiento para el personal de Casa Monarca. La interfaz se construyó empleando lenguajes nativos de la web y la API estándar fetch(), garantizando compatibilidad universal y estabilidad a largo plazo.

3.2. Gestión de identidades y política de control de acceso

El diseño de la solución contempla la dinámica de rotación de colaboradores en Casa Monarca, estructurando un modelo jerárquico para categorizar a los usuarios. Este modelo se implementó en el código a través de la enumeración LevelEnum, en donde se clasifican las identidades determinando qué perfiles requieren credenciales criptográficas y estableciendo sus límites de operación.

De acuerdo con las reglas de negocio establecidas para la Organización Socio Formadora, el flujo de la información exige un control estricto sobre quién registra, revisa y modifica los datos de los beneficiarios, así como quién posee la facultad de emitir comunicaciones oficiales en nombre de la institución. Para abordar estas dos dimensiones, el sistema modela un Control de Acceso Basado en Roles (RBAC) fundamentado

tanto en las operaciones de bases de datos CRUD (Crear, Leer, Actualizar, Eliminar) como en las capacidades de firma criptográfica.

A nivel de código, esta jerarquía se encuentra parametrizada a través del enumerador `LevelEnum`, estableciendo cuatro niveles con capacidades escalonadas:

1. **Nivel 1 — Administrador:** Posee control absoluto sobre el sistema, incluyendo la gestión completa del padrón de usuarios, la emisión de certificados, la revocación y la firma de comunicaciones institucionales.
2. **Nivel 2 — Coordinador:** Personal con autoridad para la gestión de su área. El código refleja la segmentación de este nivel mediante el enumerador `DeptEnum`, mapeando las áreas operativas de Casa Monarca. Los coordinadores poseen capacidades de firma de correos institucionales.
3. **Nivel 3 — Operativo:** Personal enfocado en la consulta de información personal y el ingreso de datos demográficos básicos. No participa en flujos criptográficos.
4. **Nivel 4 — Captura:** Personal de registro con acceso restringido a su propio perfil. No participa en flujos criptográficos.

A continuación, en el Cuadro 3.2, se sintetiza la matriz de acceso CRUD sobre el padrón de usuarios para cada perfil del sistema. Esta matriz se aplica simétricamente tanto en la capa de presentación (donde las opciones no disponibles se ocultan o deshabilitan) como en la capa de lógica del backend, que valida cada solicitud y rechaza con un código HTTP 403 cualquier operación no autorizada según el rol del solicitante.

Nivel	Perfil	Crear (C)	Leer (R)	Actualizar (U)	Eliminar (D)	Alcance Operativo
Nivel 1	Administrador	✓ Cual- quier nivel	✓ Todo	✓ Cual- quier usuario	✓	Gestión total.
Nivel 2	Coordinador	✓ Solo N3 y N4	✓ Todo	✓ N3 y N4 del mismo depto.	×	Gestión segmentada por departamento.
Nivel 3	Operativo	×	✓ Perfil propio	×	×	Consulta de perfil propio.
Nivel 4	Captura	×	✓ Perfil propio	×	×	Consulta de perfil propio.

Cuadro 3.2: Matriz de permisos CRUD sobre el padrón de usuarios (elaboración propia).

Adicionalmente, dado que la firma criptográfica constituye una facultad institucional con implicaciones legales (no repudio), el sistema diferencia explícitamente qué niveles pueden emitir comunicaciones oficiales y revocar credenciales. Esta segmentación se sintetiza en el Cuadro 3.3.

Nivel	Perfil	Recibe certifica- do	Firma correos	Revoca certifica- dos
Nivel 1	Administrador	✓	✓	✓
Nivel 2	Coordinador	✓	✓	×
Nivel 3	Operativo	×	×	×
Nivel 4	Captura	×	×	×

Cuadro 3.3: Matriz de capacidades criptográficas y firma institucional por nivel de acceso (elaboración propia).

Esta segmentación implementa el principio de mínimo privilegio descrito en el marco de referencia [Stallings, 2017, Zúñiga and Hecht, 2023]: cada usuario recibe únicamente las capacidades estrictamente necesarias para su rol institucional. La concentración de capacidades criptográficas en los niveles N1 y N2 responde a una decisión deliberada de Casa Monarca, según la cual la facultad de emitir comunicaciones oficiales con valor probatorio queda reservada al personal directivo y de coordinación, mientras que el personal operativo y de captura interactúa con el sistema exclusivamente para tareas de consulta o ingreso de datos básicos.

3.3. Desarrollo de componentes BackEnd

El servidor central (BackEnd) es el núcleo lógico y de seguridad del sistema. Su desarrollo se estructuró en seis módulos fundamentales para garantizar una separación adecuada de responsabilidades: orquestación e inicialización del servidor, interfaz REST de comunicación, autenticación y gestión de sesiones, operaciones criptográficas sobre certificados, criptografía de firma de correos, y persistencia de datos.

A continuación, se detalla la implementación técnica de cada componente.

Módulo de inicialización (`run.py`)

Este componente es el punto de entrada de la aplicación. Su responsabilidad principal consiste en parsear los argumentos de línea de comandos (archivo de base de datos, certificado del servidor y llave privada del servidor), validar la existencia de los tres archivos antes de arrancar, configurar el entorno Flask y registrar las rutas del sistema.

El servidor expone dos espacios de URL diferenciados: las rutas HTML (`/`, `/login`, `/change-password`, `/verify/<token>`) que sirven las plantillas de la capa de presentación, y el espacio `/api/*` donde reside la API REST gestionada por la extensión flask-restx. La documentación interactiva autogenerada se expone bajo `/api/docs` mediante Swagger UI.

En cuanto a la configuración de red, se habilitó el soporte para el intercambio de Recursos de Origen Cruzado (CORS), permitiendo la comunicación fluida con la capa de presentación. La terminación de las conexiones TLS se delega al servidor frontal del hosting (HostGator), el cual soporta nativamente certificados de Let's Encrypt según se documenta en la matriz de compatibilidades. Esta delegación responde a la práctica estándar de despliegue de aplicaciones Flask en entornos de alojamiento compartido, donde la capa de aplicación opera tras un proxy inverso que termina las conexiones cifradas.

Interfaz de programación de aplicaciones (`api_intf.py`)

Este módulo define la comunicación del sistema mediante una arquitectura RESTful, utilizando la extensión flask-restx. Se encarga de procesar las solicitudes entrantes, validar la integridad de las cargas útiles (*payloads*), aplicar el control de autorización por rol, y devolver las respuestas HTTP correspondientes.

Para este módulo se programaron enumeraciones estrictas para modelar la estructura organizacional de Casa Monarca. Se definieron los departamentos y los niveles jerárquicos como se detalla en el Cuadro 3.4.

Estructura (Enum)	Valores Permitidos (Colección)	Propósito Operativo
DeptEnum	Humanitaria, PsicoSocial, Legal, Comunicación, Almacén, TI	Delimitación estricta de las áreas y departamentos funcionales de Casa Monarca.
LevelEnum	admin, coordinador, operativo, captura	Asignación del nivel de acceso jerárquico asociado al ciclo de vida del usuario.

Cuadro 3.4: Estructura de Enums y valores permitidos para el control operativo (elaboración propia).

El módulo agrupa los endpoints del sistema en cuatro familias funcionales:

- **Endpoints de identidades (/users):** implementan las operaciones CRUD completas sobre el padrón de usuarios. Las peticiones GET soportan un motor de búsqueda por expresiones regulares y rangos de fechas. Para el registro (POST), se integraron validaciones estrictas, asegurando que el formato del correo electrónico sea válido mediante la biblioteca `validators` y que este sea único en todo el padrón. El POST y el PATCH delegan a un módulo auxiliar la verificación del rol del solicitante a partir del token JWT del encabezado, rechazando con código 403 cualquier operación incompatible con la jerarquía descrita en la sección anterior. Como subproducto de la creación de un usuario, el sistema genera automáticamente una contraseña temporal criptográficamente segura y la devuelve en la respuesta para que el administrador la comparta con el nuevo colaborador.
- **Endpoints de certificados (/certs):** a diferencia del flujo de la etapa anterior, el endpoint POST de esta familia ya no genera certificados internamente, sino que recibe certificados X.509 en formato PEM emitidos externamente mediante la herramienta de línea de comandos descrita más adelante. El endpoint decodifica el PEM, delega al módulo criptográfico la extracción de los atributos del certificado, verifica que dichos atributos coincidan con un usuario registrado en la base de datos, y valida la firma del certificado contra la lista de certificados administradores registrados, rechazando cualquier certificado que no esté firmado por una Autoridad Certificadora interna reconocida. El endpoint GET soporta filtrado por identificadores, rangos de fechas, estado de revocación y vigencia.
- **Endpoints de autenticación (/auth/*):** comprenden `/auth/login` para emitir tokens JWT a partir de credenciales válidas, `/auth/me` para que el cliente consulte la información de la sesión activa, `/auth/change-password` para el cambio obligatorio o voluntario de contraseña, y `/auth/logout` para cerrar sesión del lado cliente. La validación de credenciales delega al módulo `auth.py` la verificación de contraseñas y la emisión de tokens.
- **Endpoints de correos firmados (/mail/* y /public/verify/*):** permiten redactar correos firmados (POST `/mail`), consultar el padrón de correos del sistema (GET `/mail`), verificar la firma de un mensaje específico (GET `/mail/<id>/verify`), y, especialmente, ofrecer una verificación pública para destinatarios externos sin necesidad de credenciales (GET `/public/verify/<token>`). Esta última familia se describe en mayor profundidad en el módulo `mail_crypto.py`.

Módulo de autenticación (`auth.py`)

Este módulo encapsula las funciones criptográficas y de gestión de credenciales del sistema de inicio de sesión. Se apoya en la biblioteca `bcrypt` para el resguardo de contraseñas, en `PyJWT` para la emisión y verificación de tokens JWT, y en `python-dotenv` para la carga de la llave secreta desde un archivo de configuración aislado del repositorio.

El hashing de contraseñas utiliza el algoritmo `bcrypt` con un factor de costo configurable de 12 rondas. Este valor establece $2^{12} = 4096$ iteraciones internas del algoritmo, equilibrando seguridad y rendimiento sobre el hardware estándar del entorno de despliegue: cada operación de verificación toma aproximadamente entre 50 y 200 milisegundos, una latencia imperceptible para el usuario legítimo pero suficiente para encarecer drásticamente los ataques de fuerza bruta ante un eventual compromiso de la base de datos.

Para la generación de contraseñas temporales asignadas a nuevos usuarios durante el registro, el módulo utiliza el módulo `secrets` de la biblioteca estándar de Python sobre un alfabeto cuidadosamente seleccionado de 56 caracteres que excluye glifos visualmente ambiguos como `i`, `l`, `o`, `0`, `1`, `I`, `L` y `O`. Esta selección reduce el riesgo de errores cuando el administrador comunica la contraseña al usuario por canales no electrónicos. Las contraseñas resultantes tienen una longitud fija de 12 caracteres, lo que proporciona un espacio de búsqueda de $56^{12} \approx 5,7 \times 10^{20}$ combinaciones posibles, suficiente para resistir cualquier intento de adivinación durante el corto período de vida útil de la credencial temporal.

La emisión de tokens JWT se realiza con el algoritmo HS256 y una vigencia de 8 horas desde la emisión. El *payload* del token incluye el identificador del usuario, su nivel jerárquico, su departamento, y las marcas temporales de emisión y expiración. Aunque el token transporta el nivel y departamento del usuario, los endpoints sensibles del sistema realizan adicionalmente una consulta al padrón para obtener el estado actual del usuario, evitando así que un cambio de rol o departamento del lado servidor quede invalidado únicamente al expirar el token original. La llave secreta para la firma de tokens se carga al iniciar el módulo

desde la variable de entorno `JWT_SECRET`: si dicha variable no está definida, el módulo aborta con un error explícito, evitando la ejecución del servidor sin un secreto criptográfico válido.

Módulo de criptografía sobre certificados (`ccore.py`)

Este componente alberga las primitivas criptográficas relacionadas con los certificados X.509 registrados en el sistema. En la etapa anterior, este módulo era responsable de la generación interna de certificados: en la presente etapa, su rol se redefinió hacia la extracción de atributos y la verificación de cadenas de confianza, en concordancia con el flujo de emisión externa de certificados.

El módulo expone dos funciones principales. La primera, `extract_user_data`, recibe un objeto de certificado y extrae los atributos relevantes del sujeto X.509: el nombre, el correo electrónico, el identificador único del usuario y la combinación de departamento y nivel codificada en la Unidad Organizacional. Esta función actúa como puente entre los datos embebidos en el certificado y la estructura de usuario manejada por la base de datos: cuando un administrador registra un nuevo certificado en el sistema, esta función permite verificar que los atributos del documento coincidan con los del usuario al que se pretende asociar.

La segunda función, `verify_cert`, recibe un certificado y una lista de certificados de confianza (los emitidos a los administradores registrados), y verifica la firma del certificado contra cada uno de los confiables, devolviendo afirmativo si alguno valida correctamente. Esta verificación utiliza el esquema RSA-PKCS1v15 con el algoritmo de resumen embebido en el propio certificado (en el sistema actual, SHA-256), garantizando que únicamente los certificados emitidos por una Autoridad Certificadora interna reconocida puedan registrarse en el sistema.

Módulo de criptografía sobre correos (`mail_crypto.py`)

Este módulo es completamente nuevo en la presente etapa y constituye el núcleo criptográfico del sistema de firma de comunicaciones institucionales. Provee las primitivas necesarias para canonicalizar mensajes, firmar contenido con llaves privadas, verificar firmas con certificados públicos, calcular huellas digitales y codificar datos binarios para su transporte en formato JSON.

La función `canonicalize_body` resuelve un problema sutil pero crítico en la firma de mensajes a través de una red heterogénea. La firma criptográfica opera sobre una secuencia de bytes específica, por lo que el cliente que firma y el servidor que verifica deben producir exactamente la misma representación binaria del contenido a firmar, hasta el último byte. Diferencias invisibles para el usuario (saltos de línea Windows CRLF frente a Unix LF, codificaciones distintas, espacios al final) pueden invalidar una firma legítima. Para evitar esta inconsistencia, la función normaliza todos los saltos de línea a LF, ensambla un encabezado fijo con los campos `To`, `Subject` y el cuerpo, asegura que el resultado termine en un salto de línea, y codifica el resultado en UTF-8. Esta canonicalización se implementa idénticamente del lado del cliente (en el navegador, mediante JavaScript) y del lado del servidor (en Python), garantizando interoperabilidad entre ambas partes.

Las funciones `sign_bytes` y `verify_bytes` encapsulan el esquema de firma RSA-PKCS1v15 con SHA-256, codificando la firma resultante en Base64 para su transporte en JSON. La función `cert_fingerprint_sha256` calcula la huella digital SHA-256 del certificado completo, valor que se almacena junto con cada mensaje firmado como identificador robusto de la credencial utilizada. Finalmente, las funciones auxiliares `sha256_bytes` y `b64decode_bytes` encapsulan operaciones rutinarias sobre archivos adjuntos.

La decisión arquitectónica más relevante del módulo es la separación entre firma del cuerpo y firma de archivos adjuntos: cada adjunto se firma de manera independiente, lo cual permite verificar la integridad de cada componente del mensaje por separado y simplifica la lógica de almacenamiento y verificación en la base de datos.

Bootstrap del administrador inicial

Para garantizar que el sistema sea inmediatamente operable tras su despliegue, el módulo `api_intf.py` incorpora una rutina de inicialización que se ejecuta al arrancar el servidor. Esta rutina verifica si la tabla de autenticación contiene algún registro: en caso negativo, busca en el padrón un usuario con el correo

`admin@casamonarca.mx` y, si lo encuentra (típicamente porque fue creado por el script de datos de prueba), reutiliza su identificador. Si no existe, lo crea con los atributos correspondientes al perfil de administrador del departamento de Tecnologías de la Información. En ambos casos, el sistema asigna la contraseña inicial `admin123` con la bandera de cambio obligatorio activada, asegurando que esta credencial preasignada sea sustituida inmediatamente durante el primer inicio de sesión.

Capa de datos (`db.py`)

Este módulo administra la persistencia de la información en memoria secundaria, interactuando de manera exclusiva con el motor TinyDB. La capa expone cinco tablas: `users` (padrón de usuarios), `certs` (certificados registrados), `auth` (credenciales de autenticación), `mail_messages` (correos firmados) y `mail_attachments` (archivos adjuntos firmados independientemente).

Dado que objetos como fechas nativas de Python (`date`) o los certificados `x509.Certificate` no son compatibles directamente con estructuras JSON, se desarrollaron funciones bidireccionales de serialización y deserialización. Las fechas se transforman en tuplas de enteros (año, mes, día) y los certificados se codifican como cadenas de texto PEM para su almacenamiento seguro, reconstruyéndose en objetos nativos durante la lectura.

En términos de integridad referencial y unicidad, las funciones de escritura (`add_user` y `add_cert`) están programadas con rutinas de validación interna que generan identificadores únicos aleatorios mediante reintento, previniendo colisiones de datos. La función `add_user` valida adicionalmente la unicidad del correo electrónico antes de registrar al usuario.

La tabla `mail_messages` incorpora un mecanismo de doble cuerpo: el campo `body_raw` almacena el contenido original del correo (sobre el cual se calculó la firma), mientras que el campo `body` almacena una versión enriquecida que incluye un pie de página con la URL pública de verificación. Esta separación es necesaria por razones criptográficas: la URL de verificación solo puede generarse después de crear el mensaje (porque depende de un token aleatorio asignado en ese momento), por lo que la firma no puede incluirla. Aun así, el sistema necesita mostrar al usuario un cuerpo que contenga el enlace para distribución. Mantener ambos campos permite verificar la firma sobre el contenido original mientras se presenta al lector un cuerpo enriquecido.

Para la verificación pública de correos por destinatarios externos, la tabla `mail_messages` incluye los campos `validation_token`, `expires_at` y `token_used`. Estos campos materializan un mecanismo de enlace efímero de un solo uso: al crear un mensaje dirigido a un destinatario externo, el sistema genera un token aleatorio de 128 bits, establece su expiración a 24 horas desde la creación, y lo marca como no consumido. La función `mark_token_used` marca el token como consumido la primera vez que se accede al enlace público, evitando reutilización del mismo.

Por último, la tabla `auth` almacena, para cada usuario, su hash `bcrypt` de contraseña, la bandera de cambio obligatorio, y las marcas temporales de creación y última actualización del registro. Las funciones `get_auth`, `set_auth` y `clear_must_change` encapsulan el acceso a esta tabla, manteniéndola desacoplada del resto de la lógica del sistema.

3.4. Desarrollo de FrontEnd

La capa de presentación fue diseñada con un enfoque centrado en la usabilidad y la eficiencia, garantizando que el personal de Casa Monarca pueda operar el sistema sin requerir conocimientos técnicos avanzados. Su construcción se basó enteramente en tecnologías web nativas (HTML5, CSS3 y JavaScript), evitando deliberadamente marcos de renderizado como React o Angular que exigirían pasos de compilación incompatibles con el contexto de mantenimiento del socio formador.

En esta etapa, la capa de presentación creció desde un único archivo a un conjunto de cuatro plantillas, cada una correspondiente a una pantalla funcional del sistema. La Tabla 3.5 sintetiza las plantillas y su rol.

Plantilla	Función
login.html	Pantalla de autenticación: captura de credenciales, validación de campos vacíos en cliente, redirección condicional según el estado del usuario.
change_password.html	Cambio obligatorio de contraseña en el primer acceso, con validaciones independientes sobre contraseña actual, longitud mínima y coincidencia de confirmación.
index.html	Panel principal autenticado: dashboard de métricas, directorio de usuarios, redacción de correos firmados y consulta de mensajes del padrón institucional.
verify.html	Página pública de verificación de correos firmados, accesible sin credenciales mediante un enlace efímero de un solo uso.

Cuadro 3.5: Plantillas que componen la capa de presentación del sistema (elaboración propia).

Pantalla de inicio de sesión (login.html)

La pantalla de autenticación es el único punto de entrada al panel privado del sistema. Captura el correo institucional y la contraseña del usuario, aplica una validación local previa al envío para evitar peticiones innecesarias al servidor cuando los campos están vacíos, y delega la verificación de credenciales al endpoint `POST /api/auth/login` del backend. En caso de éxito, almacena el token JWT recibido y el resumen del usuario en `sessionStorage` (más adecuado que `localStorage` por su limpieza automática al cerrar el navegador), e inspecciona la bandera `must_change_password` del response para redirigir al usuario a la pantalla de cambio obligatorio o directamente al panel principal según corresponda. Los errores del servidor se presentan de forma diferenciada según su tipo: campos vacíos detectados localmente se señalizan resaltando los campos en rojo, mientras que las credenciales rechazadas por el backend se muestran en un mensaje descriptivo sobre el formulario.

Pantalla de cambio obligatorio de contraseña (change_password.html)

Esta pantalla implementa el flujo de transición desde una contraseña preasignada (la inicial del administrador o la temporal generada al registrar a un usuario) hacia una contraseña personal. La interfaz aplica cuatro validaciones independientes antes de enviar la solicitud al servidor: la presencia de la contraseña actual, una longitud mínima de ocho caracteres en la nueva contraseña, que la nueva contraseña sea distinta de la actual, y que el campo de confirmación coincida con la nueva contraseña. Cada validación falla con un mensaje específico sobre el campo correspondiente, evitando la confusión típica de un único mensaje genérico. La política de longitud mínima de ocho caracteres sin requerimientos de composición se alinea con las recomendaciones modernas del NIST SP 800-63B [Grassi et al., 2017], que desaconsejan reglas de complejidad arbitrarias por favorecer contraseñas predecibles. Tras el cambio exitoso, el sistema actualiza el hash bcrypt en la base de datos, elimina la bandera de cambio obligatorio y redirige al panel principal.

Panel administrativo (index.html)

El panel principal centraliza la operación cotidiana del sistema para los niveles administrativo y de coordinación. Al cargarse, ejecuta una rutina de verificación de sesión que confirma la presencia de un token JWT válido en `sessionStorage`, y en caso contrario, redirige al usuario a la pantalla de inicio de sesión antes de renderizar contenido alguno. Para evitar el parpadeo visual asociado al renderizado previo a la verificación, el cuerpo del documento se carga inicialmente con visibilidad oculta y se revela solo tras confirmar la autenticación. Todas las peticiones subsecuentes al backend se canalizan a través de una función auxiliar (`apiFetch`) que adjunta automáticamente el token JWT al encabezado de autorización y centraliza el manejo de respuestas: ante un código 401, la función limpia la sesión local y redirige nuevamente al

inicio de sesión.

El panel se organiza en torno a varios componentes funcionales. En primer lugar, el tablero de control superior expone cuatro tarjetas estadísticas con métricas en tiempo real del sistema: total de usuarios registrados, certificados vigentes, certificados revocados y certificados próximos a expirar en un horizonte de 30 días. En segundo lugar, una tabla dinámica del directorio de usuarios permite la búsqueda por coincidencia de texto, el filtrado por nivel de acceso y el filtrado por estado del certificado, actualizando el Modelo de Objetos del Documento (DOM) sin recargar la página. En tercer lugar, un conjunto de modales superpuestos aísla las operaciones críticas: el modal de creación con su formulario de registro, el modal de edición con datos precargados, el visor de certificados con el detalle X.509 y la barra de progreso de vigencia, el modal especializado que presenta la contraseña temporal generada automáticamente al registrar a un nuevo usuario, y el modal de carga de certificado por arrastrar y soltar para asociar archivos PEM emitidos externamente.

Para la aplicación de la política de control de acceso, el panel implementa una función centralizada `canDo(acción, usuarioObjetivo)` que evalúa si la operación solicitada es compatible con el rol del usuario en sesión y, cuando aplica, con el del usuario objetivo. Esta función actúa como fuente única de verdad para la visibilidad de botones, opciones del selector de niveles, columnas de la tabla y secciones completas de la interfaz. Cuando el usuario en sesión pertenece al nivel operativo o de captura, el panel se reconfigura automáticamente para mostrar únicamente su propio perfil, ocultando las estadísticas globales y los controles de gestión.

Panel de correos firmados

Incluido dentro de `index.html` como sección lateral del panel principal, el módulo de correos firmados permite a los usuarios autorizados (administradores y coordinadores) redactar comunicaciones institucionales con firma digital y consultar los mensajes registrados en el sistema. La tabla principal del módulo muestra el remitente con su identificador de dirección (ENVIADO o RECIBIDO), el destinatario, el asunto, la fecha, el estado de la firma evaluado por el backend, el número de adjuntos, una porción de la huella digital del certificado firmante, y los controles de acción. El alcance de visualización se ajusta automáticamente según el rol: el administrador visualiza todos los correos del sistema, mientras que el coordinador accede únicamente a los mensajes en los que participa como remitente o destinatario.

El formulario de redacción integra la composición del mensaje y la firma criptográfica en un único flujo. El usuario captura el destinatario, asunto y cuerpo del mensaje, opcionalmente adjunta archivos, y pega su llave privada en formato PEM en el área correspondiente. A medida que el usuario escribe el destinatario, la interfaz consulta el padrón local de usuarios y muestra una etiqueta diferenciada que indica si se trata de un destinatario interno (registrado en el gestor) o externo (cualquier otro correo). Esta distinción anticipa el comportamiento posterior del sistema: los correos a destinatarios externos generan un enlace efímero de verificación, mientras que los internos se verifican exclusivamente desde el panel autenticado.

La firma criptográfica ocurre íntegramente en el navegador del firmante mediante la biblioteca `jsrsasign`, que provee primitivas de criptografía RSA compatibles con los esquemas estándar utilizados por el backend. La llave privada introducida por el usuario nunca se transmite al servidor: el navegador la utiliza localmente para calcular las firmas RSA-PKCS1v15 con SHA-256 sobre el cuerpo canonicalizado y sobre cada archivo adjunto, y posteriormente envía al servidor el contenido del mensaje junto con las firmas resultantes en formato Base64. Esta arquitectura materializa el principio criptográfico de no centralización de secretos: la facultad de firmar en nombre de un usuario permanece exclusivamente bajo su control físico de la llave privada.

La canonicalización del cuerpo del mensaje implementada en JavaScript es una réplica exacta byte por byte de la función equivalente del módulo `mail_crypto.py` del backend: ambas normalizan los saltos de línea a Unix LF, ensamblan un encabezado con los campos To y Subject, aseguran que el resultado termine en un salto de línea y lo codifican en UTF-8. Esta replicación garantiza que la firma calculada en el navegador pueda ser verificada por el backend sin discrepancias originadas en diferencias invisibles de representación.

Página pública de verificación (verify.html)

La página pública de verificación constituye la interfaz que el sistema ofrece a destinatarios externos para constatar la autenticidad de los correos firmados que reciben. Es la única pantalla del sistema accesible sin credenciales, y se carga al abrir un enlace de la forma `/verify/<token>`, donde el token es el identificador efímero generado al enviar el mensaje original.

Al cargarse, la página extrae el token de la URL e invoca al endpoint público `GET /api/public/verify/<token>`. El backend ejecuta la verificación criptográfica completa: valida la firma del cuerpo del mensaje contra el certificado registrado del firmante, verifica la vigencia y el estado de revocación del certificado, marca el token como consumido para impedir reutilización, y devuelve un resumen estructurado con los resultados. La página renderiza este resultado en una tarjeta dividida en secciones: un banner superior con el veredicto general (firma válida o inválida), un bloque de estado de verificación con indicadores individuales para la integridad del mensaje y la vigencia del certificado, los metadatos del mensaje (asunto, destinatario, fecha de creación, identificador del firmante) y la huella digital del certificado firmante. En caso de token expirado, ya consumido o no encontrado, la página despliega un mensaje específico sin exponer información alguna del mensaje original, protegiendo la confidencialidad del contenido ante accesos no contemplados.

La verificación criptográfica de esta página se ejecuta del lado del servidor, no del cliente. Esta decisión de diseño prioriza la accesibilidad para destinatarios externos: el receptor puede confirmar la autenticidad de un correo desde cualquier navegador sin necesidad de descargar el certificado público de la Autoridad Certificadora interna, instalar software adicional o conocer los detalles del protocolo de firma. Como contrapartida, esta arquitectura sitúa la confianza en el servidor de Casa Monarca como verificador autoritativo (una iteración futura del sistema podría complementar este flujo con verificación opcional del lado cliente, descargando el certificado raíz y validando localmente la firma para usuarios que requieran independencia verificativa).

3.5. Conexión y flujos de operación

La interconexión y la orquestación del sistema operan mediante un modelo desacoplado de transferencia de estado representacional (REST). La comunicación externa entre la capa de presentación (FrontEnd) y la lógica de negocio (BackEnd) se realiza de forma asíncrona mediante peticiones HTTP estructuradas en formato JSON, garantizando un acoplamiento débil entre componentes. Internamente en el servidor, los módulos de la API, los componentes criptográficos y la capa de persistencia se integran de manera síncrona mediante llamadas nativas de funciones.

Con el objetivo de mitigar el ingreso de datos corruptos y garantizar la consistencia operacional, el diseño implementa una estrategia de validación ejecutada tanto en la frontera del cliente como en la del servidor. Esta doble validación cumple dos roles complementarios: del lado del cliente, ofrece retroalimentación inmediata al usuario y evita peticiones innecesarias al servidor. Del lado del servidor, por otro lado, garantiza la integridad de los datos persistidos incluso ante clientes manipulados o ataques directos al API.

A continuación, se documentan los cuatro flujos operacionales principales del sistema: el registro de identidades con generación automática de credenciales temporales, el registro de certificados emitidos externamente, la autenticación de usuarios con cambio obligatorio de contraseña, y la firma y verificación pública de comunicaciones institucionales.

3.5.1. Flujos de operación principales

Registro de nuevos usuarios

El proceso inicia en la capa de presentación cuando el administrador o coordinador introduce los datos del nuevo colaborador en el formulario modal de registro. Antes de iniciar la transmisión, el motor cliente ejecuta validaciones de primera línea: campos obligatorios no vacíos, sintaxis válida del correo electrónico y unicidad local (verificada contra la lista de usuarios cargada en memoria). Si todas las validaciones aprueban, el cliente serializa los datos y envía una petición `POST` al endpoint `/api/users` acompañada del token JWT del usuario en sesión.

El backend, mediante el módulo `api_intf.py`, recibe la petición, decodifica el token para identificar al solicitante y valida que su nivel jerárquico autorice la operación: el administrador puede crear cualquier nivel, el coordinador únicamente niveles operativo y captura, y los demás roles reciben un rechazo con código 403. Validado el permiso, el módulo verifica la sintaxis del correo mediante la biblioteca `validators` y delega la persistencia a `db.add_user`, que valida la unicidad global del correo, genera un identificador único aleatorio y registra al usuario en la tabla correspondiente.

Como subproducto del registro exitoso, el módulo `auth.py` genera automáticamente una contraseña temporal mediante un generador criptográficamente seguro, calcula su hash `bcrypt`, y registra la credencial en la tabla `auth` con la bandera de cambio obligatorio activada. La respuesta al cliente incluye el identificador del nuevo usuario y la contraseña temporal en texto plano: este es el único momento del ciclo de vida del sistema en el que dicha contraseña existe en forma legible. La interfaz presenta esta contraseña al administrador mediante un modal especializado con un único botón de copiado, y advierte que tras cerrar la ventana la contraseña no podrá recuperarse.

En la Figura 3.1 se ilustra de manera lineal el proceso síncrono que se desencadena al dar de alta una nueva entidad dentro del endpoint de usuarios del sistema.

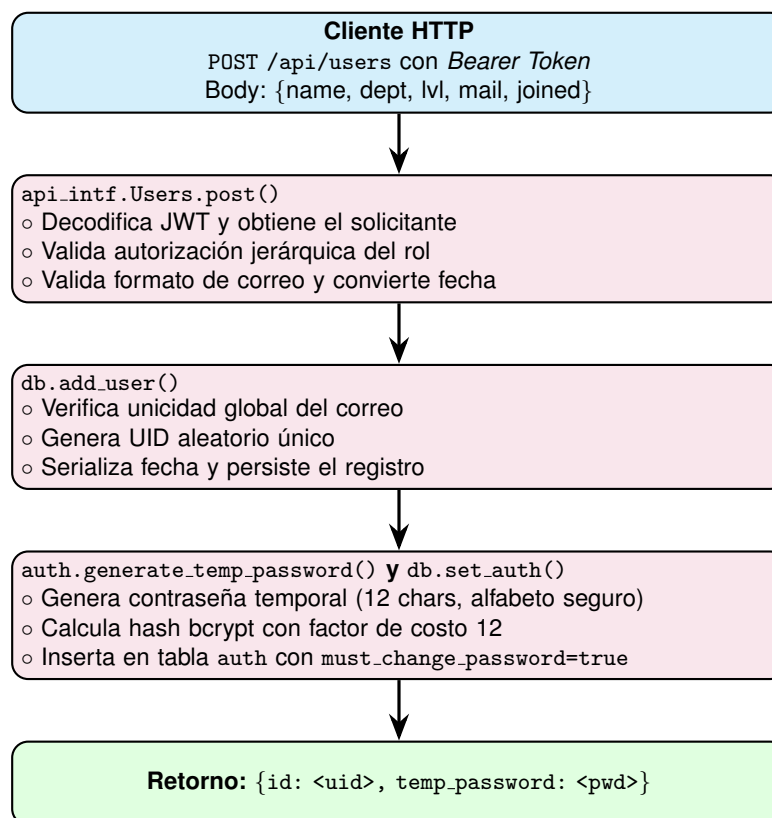


Figura 3.1: Diagrama de flujo: registro de nuevos usuarios con generación automática de contraseña temporal (elaboración propia).

Registro de certificados emitidos externamente

A diferencia del flujo de la etapa anterior, en la cual el backend generaba certificados internamente, en esta etapa los certificados se emiten fuera del sistema mediante la herramienta de línea de comandos `examples/gen_cert.py`, ejecutada localmente por el administrador. La herramienta carga la llave privada y el certificado de la Autoridad Certificadora interna, busca al usuario destinatario en la base de datos, construye el sujeto X.509 con los atributos del usuario (incluyendo el identificador y la combinación de departamento y nivel codificada en la Unidad Organizacional), genera un par de llaves RSA con tamaño configurable (mínimo 2048 bits), firma el certificado con la llave de la CA, y produce dos archivos PEM:

el certificado público y la llave privada del usuario. La llave privada debe entregarse al usuario por canal separado y nunca transita por el servidor.

Una vez emitido el certificado, el administrador lo registra en el sistema mediante el modal de carga por arrastrar y soltar que se presenta tras el alta del usuario. El cliente envía el contenido PEM al endpoint `POST /api/certs`, donde el backend ejecuta una secuencia de validaciones criptográficas. En la Figura 3.2 se detalla este proceso.

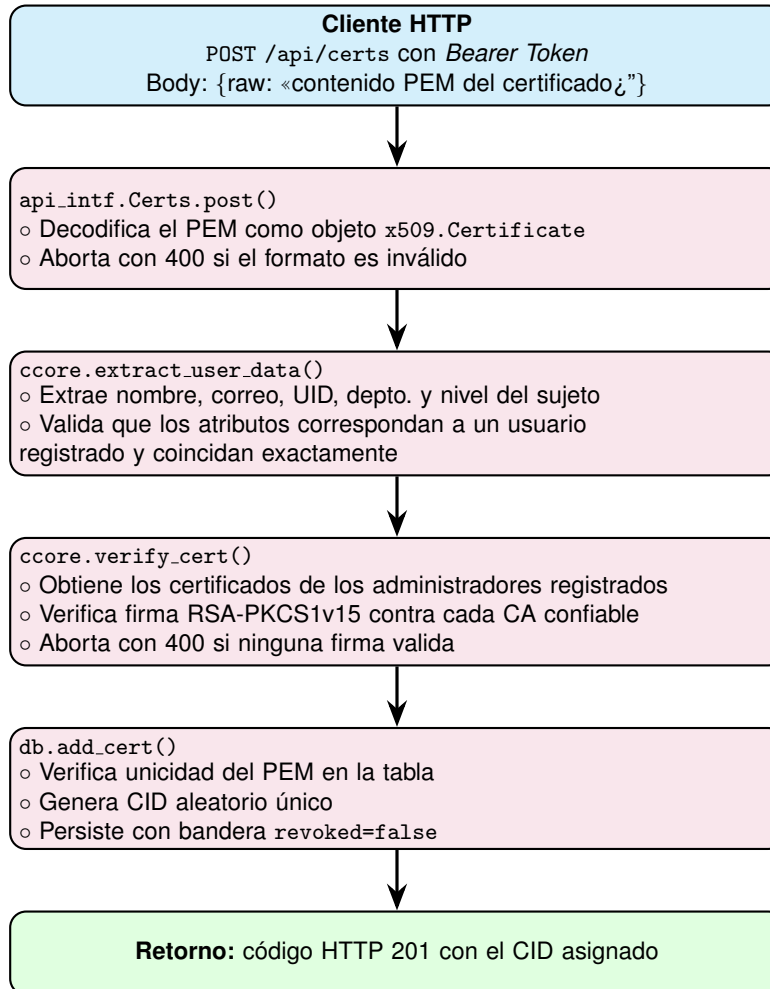


Figura 3.2: Diagrama de flujo: registro de un certificado X.509 emitido externamente por la CA interna (elaboración propia).

Posteriormente, las peticiones de consulta sobre certificados son atendidas por el endpoint `GET /api/certs`, que ofrece filtrado por identificador, usuario, vigencia temporal, estado de revocación y rango de fechas. La Figura 3.3 expone la mecánica del procesamiento de estas consultas.

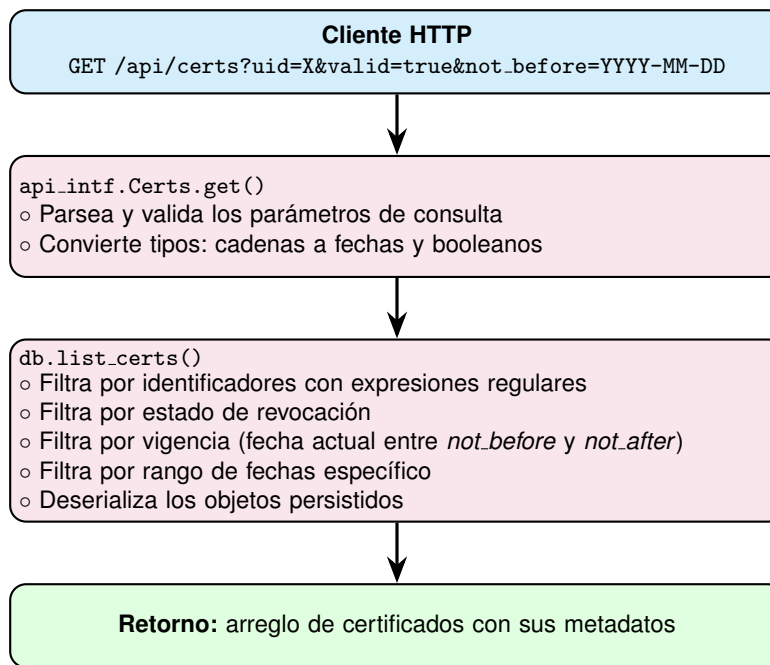


Figura 3.3: Diagrama de flujo: consulta y filtrado de certificados mediante el endpoint GET /api/certs (elaboración propia).

Autenticación y cambio obligatorio de contraseña

El flujo de autenticación cubre dos escenarios distintos pero relacionados: el inicio de sesión rutinario de un usuario cuya contraseña ya fue personalizada, y el primer acceso de un usuario que aún opera con una contraseña preasignada (la del administrador inicial o la temporal generada en el momento de su registro). El sistema discrimina entre ambos escenarios mediante la bandera `must_change_password` almacenada en la tabla `auth`.

Cuando el cliente envía sus credenciales al endpoint POST /api/auth/login, el módulo `auth.py` ejecuta la verificación contra el hash `bcrypt` registrado. Una verificación exitosa emite un token JWT firmado con HS256 y vigencia de ocho horas, cuyo *payload* incluye el identificador del usuario, su nivel jerárquico y su departamento. El cliente recibe el token junto con la bandera de cambio obligatorio y la utiliza para decidir el destino: si la bandera está activa, redirige al usuario a la pantalla de cambio de contraseña. Si no está activa, redirige al panel principal. La Figura 3.4 ilustra esta secuencia.

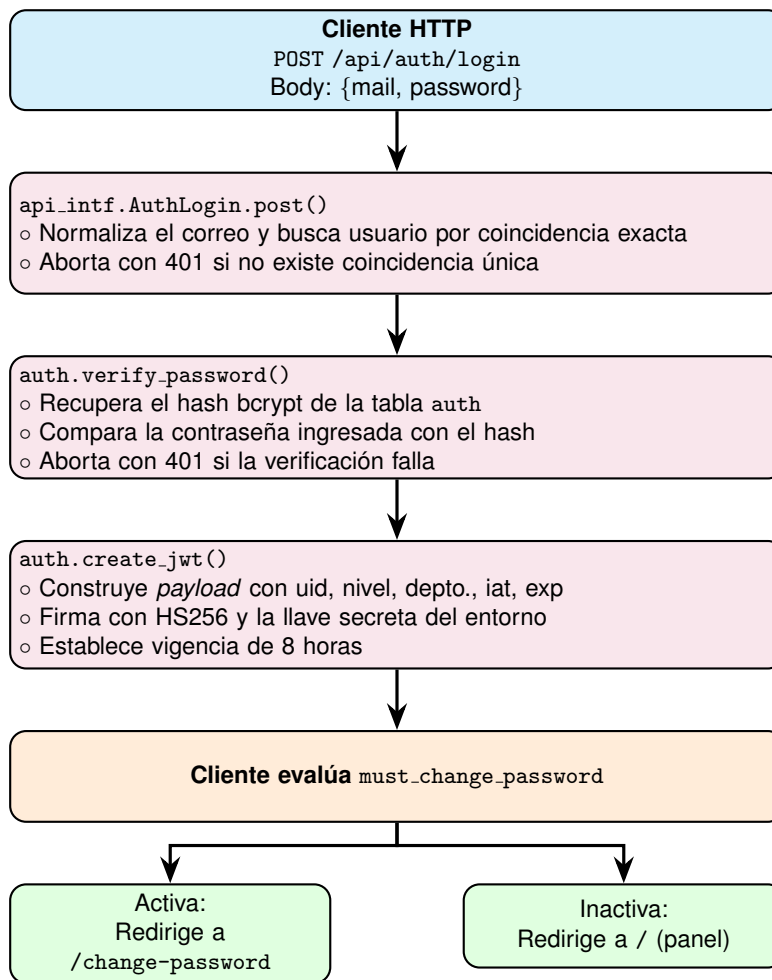


Figura 3.4: Diagrama de flujo: autenticación de usuario con discriminación de primer acceso (elaboración propia).

La pantalla de cambio de contraseña, una vez alcanzada, aplica cuatro validaciones independientes antes de transmitir la solicitud al endpoint `POST /api/auth/change-password`: la presencia de la contraseña actual, la longitud mínima de ocho caracteres en la nueva, la distinción entre la nueva y la actual, y la coincidencia entre la nueva y su confirmación. Tras la validación, el backend verifica nuevamente la contraseña actual contra el hash registrado, calcula el hash bcrypt de la nueva contraseña, actualiza la tabla `auth` y elimina la bandera de cambio obligatorio. El usuario es entonces redirigido al panel principal con su sesión ya plenamente operativa.

Firma y verificación pública de correos institucionales

El flujo de firma y verificación pública de correos constituye la funcionalidad más distintiva del sistema en la presente etapa. Combina criptografía del lado cliente para la firma, validación criptográfica del lado servidor para la verificación, y un mecanismo de tokens efímeros para permitir que destinatarios externos confirmen la autenticidad de un mensaje sin requerimientos técnicos adicionales.

El proceso comienza en el formulario de redacción del panel principal, donde el remitente captura el destinatario, el asunto, el cuerpo del mensaje, sus archivos adjuntos opcionales y su llave privada en formato PEM. La interfaz canonicaliza localmente el cuerpo del mensaje (normalizando los saltos de línea a Unix y ensamblando el encabezado fijo con campos `To` y `Subject`), calcula la firma RSA-PKCS1v15 con SHA-256 sobre el resultado utilizando la biblioteca `jsrsasign`, y firma adicionalmente cada archivo adjunto de manera individual. La llave privada nunca abandona el navegador del firmante. El cliente envía al endpoint `POST /api/mail` el contenido del mensaje junto con las firmas en formato Base64.

El backend, mediante `api_intf.MailResource.post()`, verifica que el remitente tenga nivel administrador o coordinador, que posea un certificado vigente y no revocado, y que las firmas recibidas validen correctamente contra dicho certificado utilizando `mail_crypto.verify_bytes`. Si todas las verificaciones son exitosas, el sistema discrimina el tipo de destinatario: si el correo del destinatario pertenece a un usuario registrado, el mensaje se persiste sin token de verificación pública (verificable solo desde el panel autenticado), pero si el destinatario es externo, el sistema genera un token criptográficamente aleatorio de 128 bits, establece su expiración a 24 horas, y lo asocia al mensaje. La Figura 3.5 sintetiza la secuencia.

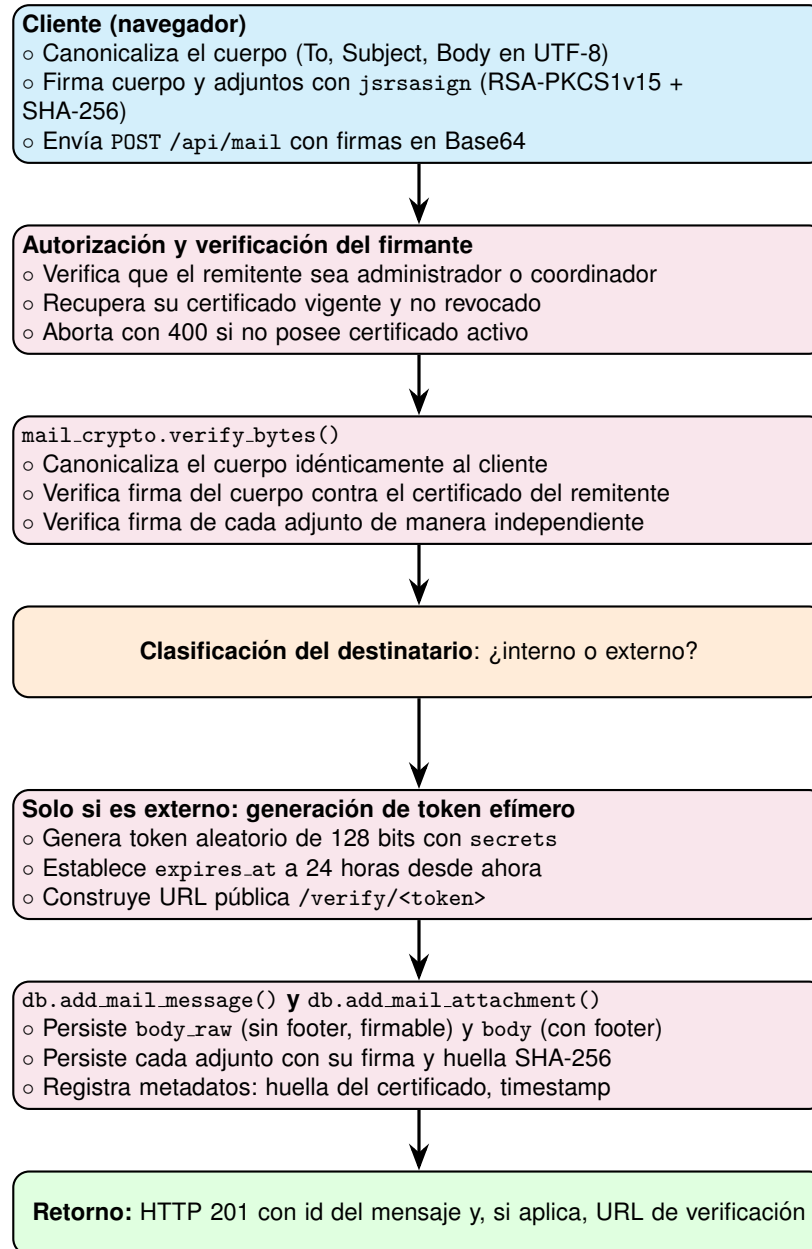


Figura 3.5: Diagrama de flujo: firma de un correo institucional y generación condicional del token efímero de verificación (elaboración propia).

Cuando el destinatario externo recibe el correo y abre el enlace `/verify/<token>`, el navegador carga la plantilla pública `verify.html`, la cual inmediatamente invoca al endpoint GET `/api/public/verify/<token>`. El backend ejecuta una secuencia particular de pasos en la cual el consumo del token precede a la verificación criptográfica, en concordancia con un modelo de confianza cero: si el token no existe, ha expirado o ya fue consumido, el sistema responde con un código de error (404 o 410 según el caso) sin exponer

información alguna del mensaje original. Esta separación entre estado del token y contenido del mensaje protege la confidencialidad del correo ante accesos no contemplados.

Si el token es válido y aún no consumido, el backend lo marca como utilizado antes de proceder con la verificación criptográfica. A continuación recupera el certificado firmante asociado al mensaje, canonicaliza el cuerpo original (almacenado como `body_raw` en la base de datos, sin el pie de página que incluye la propia URL de verificación), y verifica la firma. Adicionalmente, evalúa el estado del certificado: vigente, no revocado y dentro de su período de validez. El resultado se devuelve al cliente como un objeto estructurado con indicadores individuales por cada componente verificado, sus metadatos asociados (asunto, destinatario, identificador del firmante, huella SHA-256 del certificado) y una cadena de texto descriptiva. La plantilla `verify.html` renderiza este resultado en una tarjeta de verificación con códigos visuales diferenciados según el veredicto.

La separación entre el campo `body_raw` (sobre el cual se firma) y el campo `body` (que incluye el pie con la URL de verificación, visible para el lector pero excluido de la firma) resuelve una circularidad inherente: la URL de verificación no puede formar parte del cuerpo firmado porque depende de un token aleatorio que solo existe después de persistir el mensaje; por lo que firmar y persistir deben ocurrir antes de generar el pie. Esta arquitectura permite mostrar al destinatario un correo cuya autenticidad puede verificar sin alterar el contenido original sobre el cual se calculó la firma.

4. Resultados

Para garantizar que el sistema integrado de Gestión de Identidades y Firma de Correos cumple con los requerimientos operativos y de seguridad estipulados por la Organización Socio Formadora, se diseñó y ejecutó un plan de pruebas funcionales sobre los tres módulos principales del sistema: autenticación, gestión de usuarios y certificados, y firma con verificación pública de correos. El objetivo de esta fase fue comprobar la correcta interacción entre la interfaz de usuario (FrontEnd), la lógica de ruteo de la API (BackEnd), la integridad de las operaciones criptográficas y la aplicación del modelo de control de acceso jerárquico.

A continuación, se presenta la matriz de validación (Cuadro 4.1), la cual documenta los doce casos de prueba ejecutados, organizados por módulo, junto con el resultado esperado y el enlace a la evidencia visual correspondiente.

Clave	Módulo	Prueba	Acción del tester / usuario	Resultado Esperado	Evidencias
A-01	Autenticación	Login exitoso del administrador inicial	Iniciar sesión con admin@casamonarca.mx y admin123.	Redirige a la pantalla de cambio obligatorio de contraseña.	Ver enlace
A-02	Autenticación	Login con credenciales inválidas	Iniciar sesión con un correo registrado y una contraseña incorrecta.	El servidor responde con mensaje de error <i>Invalid credentials</i> en recuadro rojo.	Ver enlace
A-03	Autenticación	Login con campos vacíos	Pulsar <i>Iniciar sesión</i> sin capturar credenciales.	Validación local: ambos campos se marcan en rojo sin enviar petición al backend.	Ver enlace
A-04	Autenticación	Cambio obligatorio de contraseña exitoso	Capturar contraseña actual válida, nueva contraseña válida y su confirmación.	Mensaje verde de confirmación y redirección al panel principal.	Ver enlace
A-05	Autenticación	Validaciones del cambio de contraseña	Probar tres escenarios: nueva contraseña con menos de 8 caracteres, igual a la actual, y confirmación no coincidente.	Cada caso despliega su mensaje de error específico sobre el campo correspondiente.	Ver enlace
B-01	Usuarios	Crear usuario con generación de contraseña temporal	Registrar un nuevo coordinador completando todos los campos del modal.	Aparece modal con contraseña temporal generada automáticamente y botón de copia.	Ver enlace
B-02	Usuarios	Validación de correo con formato inválido	Intentar registrar un usuario con un correo sin arroba o sin dominio.	Mensaje rojo bajo el campo: <i>El correo no tiene un formato válido.</i>	Ver enlace
B-03	Usuarios	Validación de correo duplicado	Intentar registrar un usuario con un correo ya existente en el sistema.	Mensaje rojo: <i>Este correo ya está registrado para [Nombre].</i>	Ver enlace
B-04	Usuarios	Restricción de coordinador en creación de usuarios	Iniciar sesión como coordinador y abrir el modal de nuevo usuario.	El selector de niveles muestra únicamente N3 – Operativo y N4 – Captura.	Ver enlace
C-01	Certificados	Visualización del certificado de un usuario	Pulsar el ícono de visualización del certificado de un coordinador.	Modal con tarjeta de identidad, fechas de vigencia, barra de progreso y bloque PEM completo.	Ver enlace
D-01	Correos	Firma de correo a destinatario externo	Redactar correo, pegar llave privada PEM, capturar destinatario externo y pulsar <i>Firmar y enviar</i> .	Banner verde con enlace efímero de verificación. El correo aparece con la etiqueta <i>Firma válida</i> .	Ver enlace
D-02	Correos	Verificación pública del correo firmado	Abrir el enlace efímero del caso D-01 en una pestaña sin sesión activa.	Tarjeta de verificación con firma válida, certificado vigente y metadatos del mensaje.	Ver enlace
D-03	Correos	Verificación con token reutilizado o expirado	Recargar la misma URL del caso D-02 tras su primer uso.	Aviso de enlace ya utilizado, sin exposición de datos del mensaje original.	Ver enlace

Cuadro 4.1: Matriz de casos de prueba y enlaces de evidencia para los módulos de Autenticación, Usuarios, Certificados y Correos firmados. (elaboración propia).

Cada uno de los catorce casos referenciados en el Cuadro 4.1 cuenta con su evidencia visual correspondiente, hospedada en un repositorio compartido de acceso público. Adicionalmente, el manual de usuario del sistema documenta el flujo paso a paso de cada operación, sirviendo como soporte complementario a la matriz de validación.

La ejecución de las pruebas confirma la viabilidad e integridad de la arquitectura unificada propuesta. El módulo de autenticación, fundamentado en PyJWT y bcrypt, demostró la correcta emisión y verificación de tokens, así como la aplicación efectiva del flujo de cambio obligatorio de contraseña sobre credenciales temporales. La generación de contraseñas temporales sobre un alfabeto seguro de 56 caracteres y la persistencia de credenciales mediante hashing adaptativo se ejecutaron sin latencia perceptible para el usuario final.

El módulo de gestión de identidades, apoyado en la biblioteca cryptography, procesó eficientemente la verificación de certificados X.509 frente a la cadena de confianza de la Autoridad Certificadora interna. La validación cruzada del modelo de control de acceso jerárquico (RBAC) se confirmó tanto en la capa de presentación (mediante la función centralizada canDo) como en la capa de lógica del backend, que rechaza con código HTTP 403 cualquier operación incompatible con el rol del solicitante. El manejo desacoplado de la base de datos ligera (TinyDB) mantiene la integridad referencial entre las tablas users, certs, auth, mail_messages y mail_attachments al momento de ejecutar consultas con filtros combinados.

Finalmente, el módulo de firma de correos institucionales validó la correcta operación del flujo criptográfico de extremo a extremo: la firma calculada en el navegador del remitente mediante jsrsasign sobre el cuerpo canonicalizado se verificó exitosamente en el servidor, demostrando que las funciones de canonicalización implementadas en JavaScript y Python producen secuencias de bytes idénticas. El mecanismo de tokens efímeros para verificación pública por destinatarios externos operó correctamente en sus tres estados (válido, expirado y reutilizado), confirmando que la página de verificación protege la confidencialidad del contenido ante accesos no contemplados.

La estrategia de doble validación, ejecutada tanto en la frontera del cliente (para optimizar la experiencia de usuario y evitar peticiones innecesarias al servidor) como en la del servidor (para garantizar la integridad de los datos persistidos), resultó en un sistema resiliente a errores de capa humana y a clientes manipulados.

5. Conclusiones y Trabajo Futuro

La elaboración de este proyecto demostró la viabilidad de diseñar e implementar un sistema integral de gestión de identidades y comunicaciones institucionales firmadas adaptado a las restricciones de infraestructura de una Organización Socio Formadora del tercer sector. A través de una arquitectura de tres capas, se logró una plataforma centralizada que abstrae la complejidad técnica subyacente, permitiendo al personal de Casa Monarca enfocarse en su labor humanitaria sin comprometer la seguridad de la información ni sacrificar la capacidad de emitir comunicaciones con valor probatorio frente a terceros.

Desde una perspectiva técnica y disciplinar, el desarrollo de este reto permitió materializar los conceptos teóricos de las álgebras modernas y la criptografía en una herramienta de software funcional. La implementación de la criptografía de clave pública mediante el algoritmo RSA, sustentada en la intratabilidad computacional de la factorización de números primos grandes, garantizó un canal de confianza sólido para la emisión y verificación de identidades. La incorporación del esquema de firma digital RSA-PKCS1v15 combinado con la función de resumen seguro SHA-256 sobre cuerpos canonicalizados extendió esta confianza al dominio de las comunicaciones institucionales, evidenciando cómo las matemáticas aplicadas resuelven problemas críticos de autenticidad, integridad y no repudio en los sistemas de información. El uso del algoritmo de hashing adaptativo bcrypt para el resguardo de contraseñas y la emisión de tokens JWT firmados con HMAC-SHA256 para la gestión de sesiones complementan este ecosistema, materializando un sistema de autenticación moderno alineado con las recomendaciones del NIST [Grassi et al., 2017].

Una decisión arquitectónica particularmente significativa fue la determinación de que las llaves privadas de los usuarios nunca transitaran por el servidor central. Tanto la emisión de certificados (mediante una herramienta de línea de comandos ejecutada localmente por el administrador) como la firma de correos (calculada en el navegador del firmante mediante jsrsasign) operan bajo este principio, lo cual concuerda con el postulado criptográfico de no centralización de secretos y dota al sistema de garantías de no repudio robustas. Esta arquitectura sitúa la confianza del sistema en la custodia individual de las credenciales, alineando responsabilidad técnica y responsabilidad operativa de manera coherente.

En cuanto al impacto social e institucional, la solución desarrollada trasciende el ámbito tecnológico para convertirse en un mecanismo de protección a los derechos humanos. La población migrante atendida por

Casa Monarca representa un sector de extrema vulnerabilidad: por ello, asegurar las identidades digitales de quienes manejan su información, así como dotar de validez probatoria a las comunicaciones que la organización emite hacia autoridades, donantes e instituciones aliadas, resulta imperativo. El sistema entregado apoya a la organización en el cumplimiento estricto de normativas legales mexicanas como la Ley Federal de Protección de Datos Personales en Posesión de Particulares y se alinea con los principios de fiabilidad y equivalencia funcional establecidos en el Código de Comercio para los mensajes de datos firmados criptográficamente.

Especial atención merece el mecanismo de verificación pública mediante enlaces efímeros de un solo uso. Este componente extiende el alcance del sistema más allá de los muros institucionales: cualquier destinatario externo, sin credenciales en el sistema ni conocimientos criptográficos previos, puede constatar la autenticidad de un correo emitido por Casa Monarca desde un navegador convencional. Esta accesibilidad democratiza la verificación criptográfica, eliminando la barrera técnica que tradicionalmente ha mantenido a la firma digital fuera del alcance de los usuarios no especializados.

Finalmente, el sistema actual fue diseñado como un Producto Mínimo Viable que prioriza la validación funcional de la arquitectura sobre la optimización para producción. Su diseño modular y desacoplado garantiza una evolución transparente hacia un entorno de despliegue robusto: la capa de persistencia puede sustituirse por un motor relacional sin alterar la lógica criptográfica, el módulo de autenticación puede extenderse con mecanismos adicionales sin reescribir la API, y el flujo de emisión de certificados puede formalizarse en una jerarquía de Autoridades Certificadoras sin modificar el modelo de verificación. Este proyecto subraya que el uso conjunto de la ciencia de datos, la ingeniería de software y la criptografía constituye una herramienta poderosa y necesaria para potenciar, proteger y dignificar la labor de las organizaciones del tercer sector.

Trabajo Futuro

Se identifican varias líneas de trabajo prioritarias agrupadas en cuatro ejes complementarios.

Infraestructura de despliegue. El sistema actual ha sido validado en entornos de desarrollo local. Su despliegue en el entorno productivo de Casa Monarca demanda la configuración del servidor para operar bajo el modelo de proxy inverso del hosting compartido, ajustando los puntos de acceso de la capa de presentación al dominio público y deshabilitando los modos de depuración utilizados durante el desarrollo. Adicionalmente, la migración de la capa de persistencia desde el motor TinyDB hacia un sistema de gestión relacional empresarial (MySQL, soportado nativamente por el hosting de Casa Monarca) garantizará el manejo adecuado de la concurrencia y la integridad transaccional ante un mayor volumen de operaciones.

Fortalecimiento de la autenticación. Para mitigar las vulnerabilidades inherentes al factor humano (pérdida o exposición de credenciales), se priorizará la incorporación de Autenticación Multifactor mediante códigos de un solo uso basados en tiempo (TOTP), aplicables tanto al inicio de sesión rutinario como a operaciones críticas como la emisión de certificados. Complementariamente, se contempla la incorporación de mecanismos de limitación de tasa explícitos en el endpoint de autenticación y la validación de contraseñas contra listas públicas de credenciales comprometidas, alineándose plenamente con las recomendaciones del NIST SP 800-63B sobre gestión del ciclo de vida de credenciales.

Protección de credenciales criptográficas. En el alcance actual del proyecto, las llaves privadas de los usuarios se entregan en claro al destinatario, quedando bajo su custodia individual. Una iteración productiva debería incorporar el cifrado de estas llaves antes de su distribución mediante una contraseña administrativa, de modo que la pérdida o filtración del archivo PEM no comprometa por sí sola la capacidad de firma del usuario. De forma complementaria, la Lista de Revocación de Certificados deberá persistirse en el backend del sistema, propagando el estado de revocación a todas las verificaciones, incluyendo las de la página pública de verificación accesible para destinatarios externos.

Visión a largo plazo. En un horizonte temporal extendido, y de continuar el proyecto más allá de su fecha contemplada en el reto, la maduración de esta arquitectura podría sentar las bases tecnológicas para transicionar el registro inmutable de certificados emitidos y comunicaciones firmadas hacia una red descentralizada basada en tecnología Blockchain. Esta evolución ofrecería a Casa Monarca garantías reforzadas de inmutabilidad y transparencia para auditorías externas ante autoridades gubernamentales, donantes institucionales y cualquier otra parte interesada en la trazabilidad histórica de las operaciones de la organización.

Enlaces a los repositorios

1. Enlace al repositorio de Github: https://github.com/A01831983/MA2006B_Reto.git
2. Enlace a la carpeta específica de ".Entregas Finales" dentro del repositorio: https://github.com/A01831983/MA2006B_Reto/tree/main/EntregasFinales
3. Enlace a la carpeta específica ".Evidencia 1 - Reporte Técnico", dentro de ".Entregas Finales": https://github.com/A01831983/MA2006B_Reto/tree/main/EntregasFinales/2.%20Evidencia%201%20-%20Reporte%20t%C3%A9cnico

Bibliografía

- [Barker, 2020] Barker, E. (2020). NIST Special Publication 800-57 Part 1 Revision 5: Recommendation for Key Management. National Institute of Standards and Technology.
- [Cámara de Diputados del H. Congreso de la Unión, 2010] Cámara de Diputados del H. Congreso de la Unión (2010). Ley Federal de Protección de Datos Personales en Posesión de Particulares. Diario Oficial de la Federación. México.
- [Chacon and Straub, 2014] Chacon, S. and Straub, B. (2014). *Pro Git*. Apress, 2nd edition.
- [Cryptography Project, 2026] Cryptography Project (2026). *Cryptography Documentation*. Accedido en mayo de 2026.
- [GitHub, Inc., 2026] GitHub, Inc. (2026). Github features and security tools. Accedido en mayo de 2026.
- [Grassi et al., 2017] Grassi, P. A., Garcia, M. E., and Fenton, J. L. (2017). Digital identity guidelines. Technical Report Special Publication (NIST SP) 800-63-3, National Institute of Standards and Technology (NIST).
- [Jones et al., 2015] Jones, M., Bradley, J., and Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force.
- [Mohammed et al., 2025] Mohammed, A. et al. (2025). Integración continua de seguridad: Metodologías devsecops. *Software Engineering Journal*.
- [Nadipalli, 2023] Nadipalli, M. (2023). *Practical DevSecOps*. Apress.
- [OpenSSL Project, 2023] OpenSSL Project (2023). *OpenSSL Cryptography and SSL/TLS Toolkit*.
- [Oracle Corporation, 2024] Oracle Corporation (2024). *MySQL 8.0 Reference Manual*.
- [Provos and Mazières, 1999] Provos, N. and Mazières, D. (1999). A future-adaptable password scheme. In *USENIX Annual Technical Conference*.
- [Rivest et al., 1978] Rivest, R. L., Shamir, A., and Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126.
- [Secretaría de Economía, 2017] Secretaría de Economía (2017). Norma Oficial Mexicana NOM-151-SCFI-2016, Requisitos que deben observarse para la conservación de mensajes de datos y digitalización de documentos. Diario Oficial de la Federación. México.
- [Stallings, 2017] Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*. Pearson, 7th edition.
- [Zong, 2025] Zong, Y. (2025). Avances en algoritmos de curvas elípticas y funciones resumen. *Journal of Cryptographic Engineering*.
- [Zúñiga and Hecht, 2023] Zúñiga, A. and Hecht, F. (2023). Mecanismos de autorización dinámicos y granulares en sistemas modernos. *Revista de Seguridad Informática y Sistemas*.